Multiple Linear and Polynomial Regression: Theory, Data Preparation, Evaluation and Worked Examples

Rian Sengupta
riansengupta18197@gmail.com

***Abstract*—This paper presents a practical overview of multiple linear regression (MLR) and polynomial regression for predicting a numerical response from several inputs. It covers the model equations and the interpretation of coefficients, estimation by ordinary least squares using the normal equation and gradient descent, data preparation including one-hot encoding, the assumptions and diagnostics required for reliable inference, and the metrics used to evaluate fitted models. Worked examples on fuel efficiency, advertising, California housing, laptop prices and a small housing dataset illustrate the workflow. Polynomial regression is then introduced as a feature transformation within the linear-model framework, and the approaches are compared with a set of good-practice guidelines.**

***Index Terms*—multiple linear regression, polynomial regression, ordinary least squares, gradient descent, one-hot encoding, multicollinearity, model evaluation**

I. INTRODUCTION

Regression is a form of supervised learning and a statistical technique used to predict or estimate a numerical value from examples whose outcomes are already known, for instance the price of a house, an exam result, a temperature, sales or fuel economy [1]. It differs from classification in that the quantity predicted is numeric rather than a category. Simple linear regression uses a single predictor variable, whereas MLR uses two or more predictors to predict one target variable [2]. Polynomial regression adds powers of the predictor variables to a linear model [6].

These models serve two purposes: prediction and insight. Given information about a case they predict an outcome, and they show how each individual variable relates to the target while the other variables are taken into account. These insights are relative to the model and the data provided, and they do not establish causation. The choice of model depends on the data structure, the purpose of the analysis and performance on out-of-sample data.

II. BACKGROUND, EQUATIONS AND INSIGHTS

A. Simple and Multiple Linear Regression

Simple linear regression takes the form

$$y = \beta_0 + \beta_1 x + \varepsilon \tag{1}$$

where y is the dependent (response) variable, x is the independent (predictor) variable, β_0 is the intercept, i.e. the predicted value of y when $x = 0$, β_1 is the gradient, i.e. the increase in y for every one-unit increase in x, and ε is the unexplained error. MLR extends this model to include multiple predictors:

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n \tag{2}$$

The model is called linear because it is a linear combination of the inputs; it is not necessarily a straight line when all the relationships are plotted. With one predictor the model fits a line, with two predictors a plane in 3-D space, and with n predictors a hyperplane in $(n+1)$-dimensional space. Typical applications include predicting house price from area, number of bedrooms, location, age and school district; exam results from hours studied, revision classes and GPA; crop yield from fertilizer, rainfall and temperature; fuel economy from vehicle characteristics; and sales from TV, radio and newspaper advertising [2].

B. Terms and Coefficient Meaning

TABLE I
TERMS USED IN REGRESSION

Term	Meaning
Dependent variable (Y)	What we are trying to estimate or predict, e.g. house price or miles per gallon.
Independent variables ($X_1 \ldots X_n$)	Variables used to predict the dependent variable, e.g. area, income, weight, horsepower.
Intercept (β_0)	The predicted Y if all predictors were 0. It may not be useful if 0 lies outside the real range of the data.
Coefficient (β_i)	The amount Y is estimated to increase for every one-unit increase in X_i, all else being equal (*ceteris paribus*).
Residual / error	The difference between what was observed and what was predicted; the part of the outcome not explained by the model.

A positive coefficient increases the prediction and a negative coefficient decreases it, keeping everything else the same. A value close to 0 means a small estimated linear effect. Coefficients should be interpreted in meaningful units. For example, a weight coefficient of −0.0052 means about 0.0052 fewer miles per gallon (MPG) for each extra pound, or about 0.52 fewer MPG for 100 extra pounds; in the same example each extra cylinder corresponds to about 0.40 fewer MPG. The magnitudes of coefficients cannot be compared directly when the variables are in different units.

C. Matrix Form

The model can be written as a matrix equation

$$\mathrm{Y} = \mathrm{X}\beta + \varepsilon \tag{3}$$

where Y is the vector of output values, X is the design matrix (one row per observation and one column per feature), β is the vector of coefficients and ε is the error. The design matrix holds a column for each variable plus a column of ones so that the intercept is estimated alongside the other weights.

III. LEARNING THE MODEL COEFFICIENTS

A common method is ordinary least squares (OLS). For each observation in the training set the model produces a prediction and a residual, and the coefficients chosen are those that minimise the sum of squared residuals. Squaring prevents negative errors cancelling positive ones and gives more weight to larger errors. Two methods are widely used to find the minimiser.

Normal equation: a direct matrix calculation that finds the optimal coefficients in one step. It is easy to implement on smaller datasets, but the matrix computations can be slow on very large datasets.

Gradient descent: weights are initialised arbitrarily (often 0 or random), and the algorithm loops through calculating predictions, errors and gradients to move the weights in the direction that reduces error. It suits large datasets and is common in machine learning and deep learning; how quickly it converges depends on the learning rate.

When implementing gradient descent from scratch, the features are prepared and an intercept column is added; the data are split; the weights are initialised; predictions and errors are computed in a loop and the weights updated according to the learning rate; finally predictions are made on held-out data and an error metric such as the mean squared error (MSE) is calculated. The method chosen depends on dataset size and available computing power. Once the coefficients have been estimated from examples with known outcomes, the equation can predict new rows with unknown outcomes, and prediction quality must be checked on data not used to fit the model.

IV. DATA PREPARATION AND CATEGORICAL ENCODING

A. Preparing Data

Model quality depends on the quality of the inputs. The target is identified first and potential predictors are chosen. Column names, types, ranges, summary statistics and sample rows are then inspected, and relationships and correlations between features and the target are identified. Missing data must be handled by dropping rows or imputing; in the housing example, 207 rows with missing total bedrooms (approximately 1% of 20,640 rows) were dropped.

Anomalies and outliers are investigated using methods such as boxplots, since extreme values can heavily affect the fit. The reason for such values should be considered before any change is made; robust scaling is an alternative. Categorical variables are converted to numerical form, and numerical variables with very different ranges are scaled where beneficial; the housing example used RobustScaler to reduce the effect of outliers. The data are split into training and test sets, and preprocessing is fitted on the training data only and then applied to the test set, which prevents information leakage.

B. One-Hot Encoding

Regression models generally require numerical inputs, yet many datasets contain categorical variables such as gender, location, fruit type, country of origin, colour, customer comments, neighbourhood or proximity to the ocean. Assigning arbitrary numbers (Apple = 1, Mango = 2, Orange = 3) wrongly implies an order or distance between values. Where a real order exists (Low, Medium, High) label or ordinal encoding is appropriate [3].

Dummy variables are binary (0/1) variables, and one-hot encoding produces one binary variable per category, so each observation has exactly one 1 across the categories (Table II).

TABLE II
ONE-HOT ENCODING EXAMPLE

Fruit	**Apple**	**Mango**	**Orange**	**Price**
Apple	1	0	0	5
Mango	0	1	0	10
Apple	1	0	0	15
Orange	0	0	1	20

If all dummies are included alongside an intercept, perfect multicollinearity arises, known as the dummy variable trap: if Apple and Mango are both 0 the fruit must be Orange. One dummy is therefore dropped, leaving a reference (baseline) category, and the remaining dummies are interpreted relative to it. In pandas this is done with `drop_first=True`.

The advantages are that categorical variables can be used in numerical models, nominal categories receive no artificial order, and the approach is simple, interpretable and shows how categories relate to the target. The disadvantages are many more columns, sparse data with many zeros, a risk of overfitting when there are many categories and little data, and loss of any natural order. With a very large vocabulary the method becomes impractical, and it cannot express similarity between words.

Best practice is to know whether categories are nominal or ordered, choose the correct encoding, remove one dummy per variable, perform feature selection after encoding, and monitor test performance to avoid overfitting. Alternatives include ordinal/label encoding for ordered categories, binary encoding to reduce the number of columns, target encoding (replacing a category by the mean of the target) and learned embeddings, which are common in deep learning.

In Python, `pd.get_dummies(df, columns=['Gender','Remarks'], drop_first=True)` may be used; alternatively scikit-learn's `OneHotEncoder` can be used within pipelines. The categorical columns are fitted and transformed, new column names are obtained with `get_feature_names_out()`, and the result is combined with the remaining features; `select_dtypes(include=['object'])` finds the object

columns. In the housing data, one-hot encoding was applied to `ocean_proximity`.

V. ASSUMPTIONS, DIAGNOSTICS AND MULTICOLLINEARITY

TABLE III
ASSUMPTIONS OF LINEAR REGRESSION

Assumption	Meaning / diagnostic clue
Linearity	The relationship between each predictor and the target is well described as linear.
Independence	Data points and error terms are independent. Multiple readings from the same source need consideration.
Homoscedasticity	Residuals have constant variance. A funnel or varying spread indicates heteroscedasticity.
Normality of residuals	Residuals are approximately normal, allowing standard tests and confidence intervals. The input columns themselves need not be normal.
Low multicollinearity	Predictors are not overly correlated.

Multicollinearity appears when features are highly correlated, for example height and weight, or car weight and horsepower. It makes it hard to identify how much each variable contributes independently, the coefficients become unstable and unreliable, and the inputs contain redundant information. It can be detected in two common ways. A correlation matrix is inspected, where values close to ±1 are a warning and a common rule of thumb is around 0.8. The variance inflation factor (VIF) indicates how much the variance of a coefficient is inflated by its relationship with other features, and a value around 10 is a common warning. Remedies are to remove one of the correlated features and keep the most useful, to combine them into a composite feature, or to apply regularisation such as ridge regression.

Violations of the assumptions, outliers, and biased or unrepresentative data can all cause problems. Predictions outside the range of the training data are unreliable. Correlation does not equal causation: a coefficient shows association in the data, not that the feature causes the outcome. Choices made in selecting and cleaning data affect results, and biased data can lead to biased predictions that increase inequalities.

VI. BUILDING AND EVALUATING AN MLR MODEL

A. Standard Python Workflow

TABLE IV
STANDARD PYTHON WORKFLOW

Stage	Procedure
1. Prepare	Import numpy, pandas, matplotlib and sklearn. Load and explore the data; establish variables, meaning and units. Tidy missing data, examine outliers, encode categoricals, scale if required, and avoid the dummy variable trap.
2. Split	Split into training and test sets, often 80/20 or 75/25. `random_state=42` makes the split reproducible.
3. Fit	Create a linear regression model and fit it to the training data.
4. Inspect	`model.intercept_` gives β_0; `model.coef_` gives the predictor coefficients.
5. Predict	`y_pred = model.predict(X_test)`
6. Evaluate and refine	Evaluate predictions with appropriate metrics and check residuals. Remove weak or redundant variables only if justified.
7. Visualize (optional)	Plot data, fitted plane or residuals.

The data are loaded with `pd.read_csv()` and inspected with `head()` and `info()`. Features (X) and labels (y) are selected, cleaned and split, a `LinearRegression()` is fitted, and the coefficients are checked before predicting and evaluating. Pandas and NumPy handle data manipulation, Matplotlib and Seaborn data exploration, and scikit-learn splitting, fitting and evaluation [2]. Fit during training alone is not sufficient evidence of quality.

B. Evaluation Measures

TABLE V
EVALUATION MEASURES

Measure	What it tells you
R^2 (coefficient of determination)	Proportion of target variance explained by the model: 1 is a perfect fit, 0 means nothing is explained (0.9011 ≈ 90.11% explained). A high value does not guarantee an unbiased model or good future performance.
Adjusted R^2	R^2 adjusted for the number of predictors. Plain R^2 always rises with more predictors; adjusted R^2 rises only if they help, so it suits comparing models of different sizes.
MAE	Mean of absolute errors.
MSE	Mean of squared errors; large errors are penalised more; units are squared target units.
RMSE	Square root of MSE, in target units. Lower MAE, MSE and RMSE generally mean lower error.
t-statistic and *p*-value	Test whether a coefficient differs significantly from zero. If $p < \alpha$ (typically 0.05) the null hypothesis of no relationship can be rejected. A high *p* suggests reviewing the predictor.
Confidence interval	Estimate ± critical $t \times$ standard error; the range the true coefficient could take.
ANOVA	Splits total variance into explained and unexplained parts; tests whether the overall model is useful.
Residual plot	Residuals against predictions should look random; a curve indicates non-linearity and a funnel indicates heteroscedasticity.

Multiple forms of evidence should be used rather than a single metric: predictive error, adjusted R^2, coefficient significance, residual patterns, assumptions and the purpose of the model. Residuals should be inspected and training and test performance compared. Cross-validation gives more reliable performance estimates and helps detect overfitting.

VII. WORKED EXAMPLES AND APPLICATIONS

A. Fuel Efficiency

The aim is to predict MPG from weight, horsepower, acceleration, cylinders, model year and origin. Candidate quantitative variables were assessed using adjusted R^2, *t*-statistic, *p*-value and correlation. Horsepower was removed because its correlation with weight exceeded 0.8, and acceleration was removed because $p > 0.05$. The final model used weight and model year, and adjusted R^2 increased from 70.4% to 80.8%, showing that a simpler model can work well. The weight coefficient of roughly −0.0052 is equivalent to −0.52 MPG per 100 lb. In an earlier model that included cylinders, each extra cylinder was associated with a coefficient of −0.3979 (about 0.40 fewer MPG).

B. Advertising

Sales are predicted from TV, radio and newspaper spend. After loading the data, splitting, fitting and predicting, the model explained 90.11% of the variation, and the MAE, MSE and RMSE values were low.

C. California Housing

This example uses MedInc and AveRooms to predict house price with an 80/20 split. The fitted model has an intercept of about 0.597 (0.5973), a weight for income of about 0.436 (0.4363) and a weight for rooms of about −0.04 (−0.0402). Higher median income gives a higher predicted price, whereas rooms has a small negative weight and contributes little. The fitted plane can be visualised with the two predictors on the *x* and *y* axes, house value on the *z* axis, and the data points plotted. The weights vary with the variables chosen, the data used and how the data are split.

D. Housing Data Preparation

This dataset has 20,640 rows and 10 columns: nine predictors and one target (`median_house_value`). The variable `ocean_proximity` is categorical, `total_bedrooms` has 207 missing values, and there are many outliers. One-hot encoding and RobustScaler were used, and the evaluated model gave an R^2 of about 0.6472, which varies with the data used.

E. Laptop Price

Laptop price is predicted from processor speed, screen size, RAM and weight. Processor speed, screen size and RAM have positive weights while weight has a negative weight. This shows how several weighted variables are combined with the intercept into a single prediction; it is an illustration of how the equation works numerically.

F. Small Housing Dataset (Learning from Scratch)

This example has six houses with sizes 1400, 1600, 1700, 1875, 1100 and 1550; bedrooms 3, 4, 3, 5, 2 and 3; ages 10, 15, 7, 2, 20 and 13; and prices 245000, 312000, 279000, 308000, 199000 and 290000. An intercept was added, the data points were split, weights were trained using gradient descent, predictions were made and the MSE calculated, showing how the model learns.

Other applications include crop yield from fertiliser, rain and temperature; financial modelling such as stock price or credit risk; income from socio-economic characteristics; business modelling such as employee performance or revenue; house price and sales prediction; and medical research [2]–[5].

VIII. POLYNOMIAL REGRESSION

A straight line or plane may fail to capture a curved relationship. Polynomial regression models curvature by adding powers of an input as extra features [6], [7]. For one predictor, a degree-2 model is

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon \tag{4}$$

and a degree-3 model is

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \varepsilon \tag{5}$$

Although the fitted curve is non-linear in x, the model remains linear in its coefficients. Treating x, x^2 and x^3 as separate feature columns allows ordinary MLR to fit them, so polynomial regression is a feature transformation within the linear-model framework.

TABLE VI
POLYNOMIAL DEGREE AND BEHAVIOUR

Degree	**Typical behaviour**
Degree 1	Too simple for non-linear relationships.
Moderate degree	Captures trend and curvature.
Very high degree	Flexibility leads to overfitting: very good on training data but poor at predicting new data.

Appropriate evaluation metrics and a train-test split should be used to choose the optimum degree, and the degree should be kept relatively low. Scaling needs attention because powers of x can become very large or small and may be correlated. As with MLR, a model that is too complex for the amount of data starts to learn noise, so residuals, test performance and the training data range should be checked.

IX. COMPARING APPROACHES AND GOOD PRACTICE

TABLE VII
COMPARISON OF REGRESSION APPROACHES

Aspect	**Simple linear**	**Multiple linear**	**Polynomial**
Features	One independent variable	Two or more independent variables	One or more inputs expanded with powers and/or interactions
Fitted relationship	Straight line	Plane or hyperplane	Curve, while linear in coefficients
Parameters	Intercept and one slope	Intercept and one slope per predictor	Intercept and a coefficient per polynomial feature
Complexity	Lower	Higher; needs careful feature selection	Grows with degree and number of expanded features
Best suited to	Simple two-variable relationships	Outcomes influenced by several factors	Relationships where curvature matters

MLR can use both numerical and encoded categorical variables and is useful in complex processes for planning, controlling, predicting and forecasting. Too many variables can cause overfitting. Polynomial regression is useful when a linear trend is insufficient, as a polynomial curve can be a good approximation. The following practices are recommended:

1) Choose variables with care; simpler models with justified variables are preferred.
2) Check the assumptions of linearity, independence, homoscedasticity, normal residuals and low multicollinearity.
3) Examine outliers, missing values, sampling bias and unusual observations.
4) Avoid data leakage by fitting pre-processing steps on training data only.
5) Compare training and test performance, using cross-validation where appropriate.
6) Interpret coefficients in real-life terms and do not assume causality from correlation.
7) Avoid extrapolation that cannot be trusted.
8) Consider feature selection, normalisation/standardisation, regularisation (Ridge/Lasso) and stepwise selection; non-linear models such as decision trees may be considered if justified.

X. CONCLUSION

MLR predicts a single quantitative variable from several quantitative or properly coded categorical variables. Its effectiveness depends on how well the data are prepared, the features selected, the coefficients estimated and the model evaluated on new data. Coefficients can be estimated using OLS, via the normal equation or gradient descent, and their reliability can be assessed with correlation analysis, VIF, residual plots, significance tests and fit/error measures.

Polynomial regression extends linear regression by raising predictors to powers, creating curves that improve fit when a line is insufficient, though a very high degree risks overfitting. Best practice is to understand the data, verify the assumptions, keep the model simple, interpret the findings and focus on good generalisation.

REFERENCES

[1] "Understanding multiple linear regression," *Medium* (The Startup). [Online]. Available: https://medium.com/swlh/understanding-multiple-linear-regression-e0a93327e960

[2] "ML | Multiple linear regression using Python," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/ml-multiple-linear-regression-using-python/

[3] "ML | One-hot encoding," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/

[4] "Mastering multiple linear regression: A comprehensive guide," *Medium*. [Online]. Available: https://medium.com/@lomashbhuva/mastering-multiple-linear-regression-a-comprehensive-guide-f36c36404fe0

[5] "Machine learning: How to easily understand multiple linear regression," *Medium* (Data Driven Investor). [Online]. Available: https://medium.datadriveninvestor.com/machine-learning-how-to-easily-understand-multiple-linear-regression-4b2ac5f161d4

[6] "Understanding polynomial regression," *Medium* (Analytics Vidhya). [Online]. Available: https://medium.com/analytics-vidhya/understanding-polynomial-regression-5ac25b970e18

[7] "Python implementation of polynomial regression," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/python-implementation-of-polynomial-regression/